Experiment 4

k-Nearest Neighbors (KNN)

Aim

To apply the k-Nearest Neighbours (KNN) algorithm for distance-based classification, investigate the influence of the number of neighbours (k) on model behaviour, and assess its performance using appropriate evaluation measures.

Theory

1. Overview

The k-Nearest Neighbours (KNN) algorithm is a supervised machine learning algorithm that is commonly used for both classification and regression tasks. It is considered a non-parametric method because it does not assume any predefined distribution for the data. Instead of learning a mathematical model during training, KNN makes predictions by comparing new data points with the existing training samples based on their similarity. This similarity is typically measured using distance metrics such as Euclidean distance, which determine how close one data point is to another in the feature space.

The concept of nearest neighbour classification was originally introduced by Fix and Hodges and later extended by Cover and Hart. In this method, when a new test sample is given, the algorithm identifies the k nearest training samples in the dataset and assigns a class label based on majority voting among those neighbours. The value of k represents the number of neighbouring data points considered during classification and plays an important role in determining the accuracy of the model.

Unlike many other machine learning algorithms, KNN does not involve an explicit training phase. Instead, it simply stores the entire training dataset and performs calculations only when a prediction is required. For this reason, it is often referred to as a lazy learning or instance-based learning algorithm. Since KNN relies on distance calculations, feature scaling is important to ensure that all features contribute equally to the distance measurement. Without proper scaling, features with larger numerical ranges may dominate the calculation and affect the performance of the algorithm.

2. Working of KNN

The working of the KNN algorithm involves identifying the nearest neighbours of a given data point and determining its class based on those neighbours. The basic steps followed by the algorithm are:

- Choose the number of neighbours k.
- Calculate the distance between the new data point and all points in the training dataset.
- Sort the distances and identify the k nearest neighbours.
- Count the class labels of these neighbours.
- Assign the class that appears most frequently among them to the new data point.

Thus, the classification of a new data sample is determined by the majority class among its nearest neighbours.

3. Determining the value of k

The parameter k represents the number of nearest neighbors considered for classification. Choosing an appropriate value of k is important for the performance of the algorithm. If k is too small, the model becomes sensitive to noise and may lead to overfitting. If k is too large, the model may include many distant points, which can reduce accuracy. In practice, the value of k is selected by testing different values and choosing the one that provides the best classification performance for the dataset.

4. Distance Metrics in KNN

a. Euclidean Distance (L2 norm)

Euclidean distance (L2 norm) is the most commonly used distance metric in KNN, where the distance between two data points is computed as the straight-line distance in feature space. It is given by $d = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$, where x_i and y_i represent the values of the i^{th} feature of data points X and Y, and n is the total number of features. In a two-dimensional space, this simplifies to $d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$. This metric is intuitive and works well when features are continuous and on similar scales, as it captures the true geometric distance between points, making it effective for identifying the nearest neighbors in KNN classification.

$$d = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

Where,

- x_i: value of the i^{th} feature of data point X
- y_i: value of the i^{th} feature of data point Y
- n: total number of features (dimensions)
- For 2D: $d = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$

b. Manhattan Distance (L1 norm)

Manhattan distance (L1 norm) is another widely used distance metric in KNN, where the distance between two data points is calculated as the sum of the absolute differences of their corresponding feature values. It is defined as $d = \sum_{i=1}^{n} | x_i - y_i |$, where x_i and y_i denote the values of the i^{th} feature of data points X and Y, and n represents the total number of features. In a two-dimensional space, this simplifies to $d = | x_1 - y_1 | + | x_2 - y_2 |$. Unlike Euclidean distance, Manhattan distance measures movement along the axes (similar to navigating a grid), making it more robust to outliers and suitable for high-

dimensional data or scenarios where feature differences are more meaningful than direct geometric distance.

$$d = \sum_{i=1}^{n} | x_i - y_i |$$

Where,

- x_i: value of the i^{th}feature of data point X
- y_i: value of the i^{th}feature of data point Y
- n: total number of features
- For 2D: $d =| x_1 - y_1 | +| x_2 - y_2 |$

5. Non-Parametric Nature of KNN

The k-Nearest Neighbours algorithm is called a non-parametric model because it does not assume any specific mathematical distribution for the data. Instead of estimating parameters of a predefined equation, the algorithm stores the training samples and performs predictions by comparing new data points with the stored examples. Because no explicit model is learned during training, KNN is also considered an instance-based learning method where predictions are made directly using the training instances.

6. Hyperparameter Selection (Choosing the Value of k)

The value of k is a hyperparameter that determines how many neighbours participate in the classification decision. Choosing an appropriate value of $k$is important for achieving good performance.

- Small values of k (e.g., $k = 1$) may make the model sensitive to noise and lead to overfitting.
- Large values of k may smooth the decision boundary too much and lead to underfitting.

Therefore, selecting a suitable value of $k$is necessary to balance bias and variance in the model. Typically, cross-validation techniques are used to determine the optimal value of k.

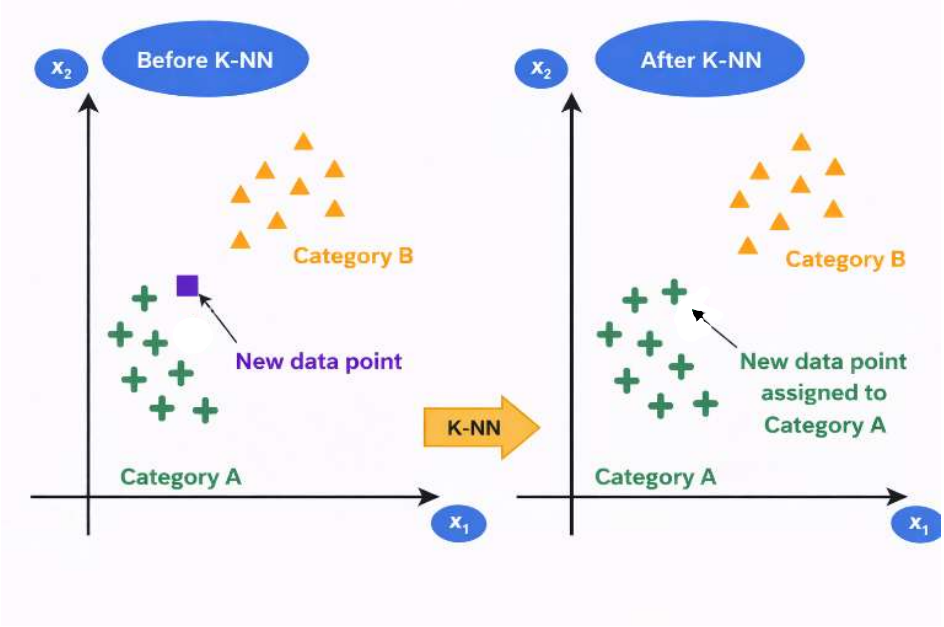


Figure 1: Illustration of K-Nearest Neighbors (K-NN) classification showing reassignment of a new data point based on nearest neighbors

The figure illustrates how the K-Nearest Neighbors (K-NN) algorithm classifies a new data point based on the majority class of its nearest neighbors. In the left panel (Before K-NN), two clusters are shown: Category A (green "+" points) and Category B (orange triangles). A new data point (shown in a different color and shape) lies between the two categories, and its class is initially unknown. In the right panel (After K-NN), the algorithm evaluates the nearest neighbors of the new data point. Since most of its closest points belong to Category A, the new data point is assigned to Category A. This is reflected by changing its color and shape to match Category A. This demonstrates how the KNN algorithm uses proximity and majority voting to classify new data samples.

7. Algorithm

Step 1: Store all training data
Step 2: Choose the value of K
Step 3: When a new data point arrives for classification:
Step 4: Calculate distance from new point to all training points
Step 5: Sort all training points by distance in ascending order
Step 6: Select the K nearest neighbours
Step 7: For Classification, use majority voting by counting how many of the K neighbours belong to each class
Step 8: For Regression, take the average of target values of K neighbours and compute Prediction as $(value_1 + value_2 + ... + value_k) / K$

8. Practical Considerations in K-Nearest Neighbors (KNN)

Handling Ties: When K neighbours have equal votes:

- Reduce K by 1 and re-vote
    - OR choose the class of the nearest neighbour among tied classes
    - OR randomly select among tied classes

Feature Scaling (Critical for KNN): *Why needed:* KNN uses distance, so features with larger ranges dominate. Before applying KNN:

- Apply Min-Max Scaling: x' = (x - min) / (max - min)
- OR Z-Score Normalization: x' = (x - mean) / std

Choosing Optimal K: *Cross-Validation Method:*

1. Split data into training and validation sets
2. For K = 1, 3, 5, 7, ... (try multiple values):
   - Train KNN with that K
   - Measure accuracy on validation set
3. Select K with highest validation accuracy

9. Merits and Demerits of k-Nearest Neighbours (KNN)

Merits of KNN	Demerits of KNN
Simple and easy to understand	High computational cost during prediction
Does not require an explicit training phase	Requires large memory to store training data
Makes no assumption about data distribution	Highly sensitive to feature scaling
Works well for small and well-separated datasets	Performance degrades in high-dimensional data
Can be used for both classification and regression	Sensitive to noise and outliers

10. Applications

The K-Nearest Neighbours (KNN) algorithm is widely used in many real-world applications because of its simplicity and effectiveness in classification and prediction tasks. Some common applications include:

- Image Recognition: KNN is used in image classification systems where images are categorized based on similarities with labelled images in the dataset.
- Recommendation Systems: It helps recommend products, movies, or music by finding users with similar preferences.
- Medical Diagnosis: KNN can assist in disease prediction by comparing a patient's data with similar cases in medical datasets.
- Pattern Recognition: It is used to recognize patterns in data such as handwriting recognition or speech recognition.
- Credit Scoring and Fraud Detection: Financial institutions use KNN to classify transactions and detect unusual or fraudulent activities.

These applications demonstrate how KNN can effectively classify data based on similarity and proximity in different domains.

KNN - Practical Implementation

Step 1: Importing Libraries

 # Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print("Imported Analysis and Plotting libraries")
```

Output:
```
Imported Analysis and Plotting libraries
```

 # Import scikit-learn modules

```
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import label_binarize
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)
print("Imported Sklearn Modules")
```

Output:
```
Imported Sklearn Modules
```

Step 2: Loading Dataset

 # Loading and Reading Data

```
iris = load_iris()
X = iris.data
y = iris.target
print("Extracted Feature and Target as numpy Arrays")
```

Output:
```
Extracted Feature and Target as numpy Arrays
```

 # extracting feature names

```
names = iris.feature_names

# extracting target names
target_names = iris.target_names

# making a dataframe with column names
iris_df = pd.DataFrame(X, columns=names)
iris_df["label"] = [target_names[i] for i in y]
iris_df.head()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

Step 3: Data Analysis

 # Data Analysis

```
# Overview
iris_df.info()
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column             Non-Null Count  Dtype
---  ------             --------------  -----
0   sepal length (cm)  150 non-null    float64
1   sepal width (cm)   150 non-null    float64
2   petal length (cm)  150 non-null    float64
3   petal width (cm)   150 non-null    float64
4   label              150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

 # Target counts

```
iris_df["label"].value_counts()
```

Output:

```
label
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```

Dataset description

```
iris_df.describe()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Finding any null values

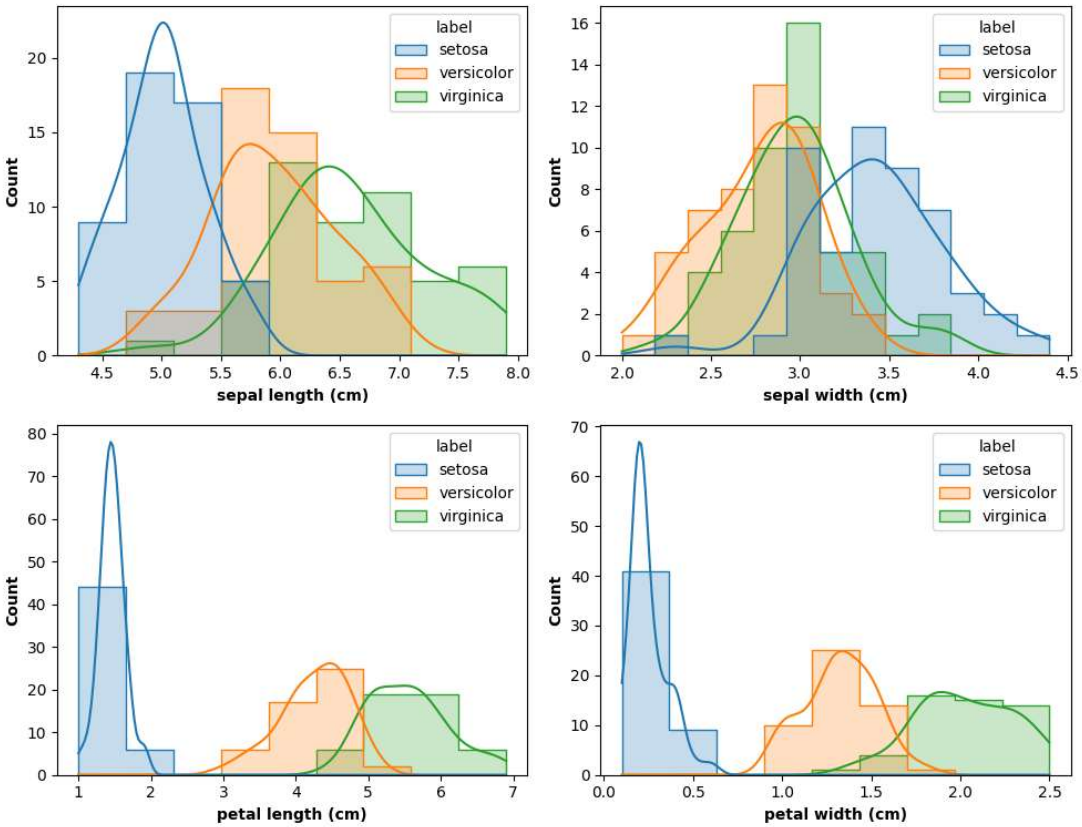
```
iris_df.isnull().sum()
```

Output:

```
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
label                0
dtype: int64
```

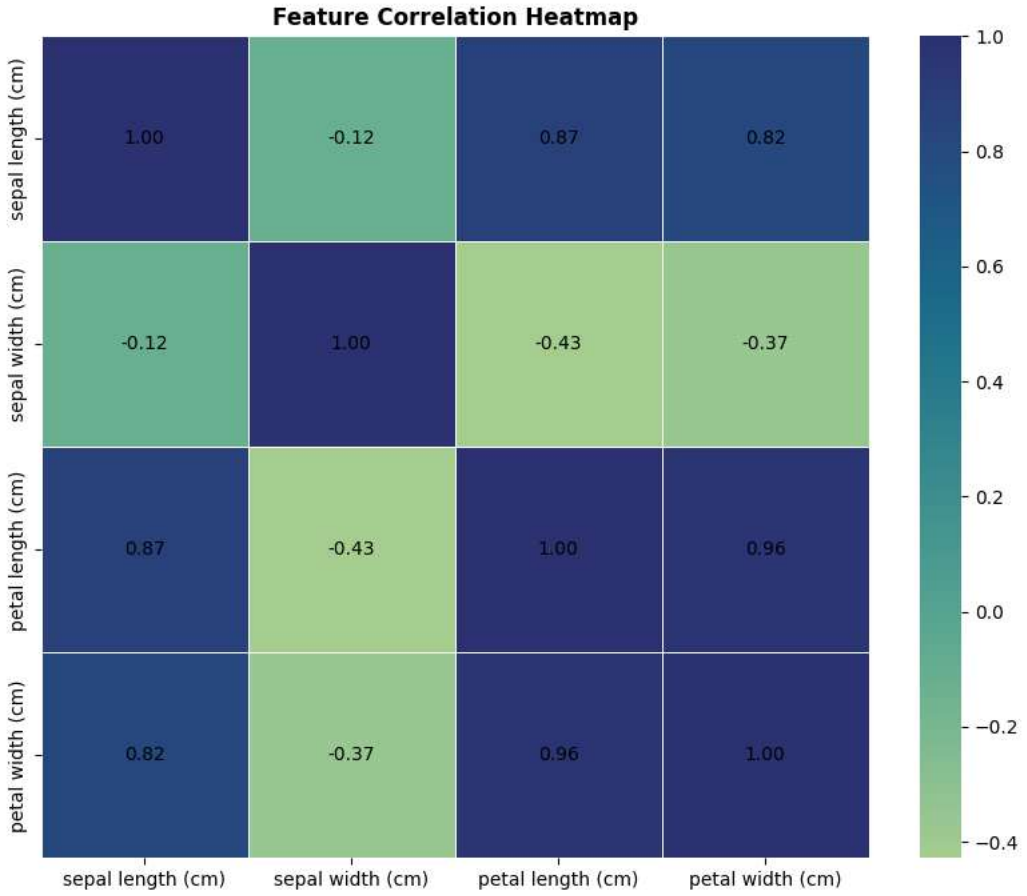
Plotting a histogram distribution of data points with labelled classes

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for ax, col in zip(axes.ravel(), names):
    sns.histplot(
    data=iris_df, x=col, hue="label",
    kde=True, element="step", ax=ax
    )
    ax.set_xlabel(col, fontweight='bold')
    ax.set_ylabel("Count", fontweight='bold')
plt.suptitle("Feature Distributions by Class", y=1.02, fontweight='bold')
plt.tight_layout()
plt.show()
```



Plotting correlation between features

```
plt.figure(figsize=(10, 8))
ax = sns.heatmap(
    iris_df.iloc[:, :4].corr(),
    annot=True,
    cmap="crest",
    fmt=".2f",
    linewidths=0.7,
    # Force annotation text to be black
    annot_kws={"color": "black"}
    )
plt.title("Feature Correlation Heatmap", fontweight='bold')
plt.show()
```



Step 4: Data Preprocessing

 # Data Preprocessing

```
# Data Encoding
# we already have encoded targets from sklearn datasets
iris_df["target"] = y
iris_df.head()
```

Output:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	label	target
0	5.1	3.5	1.4	0.2	setosa	0
1	4.9	3.0	1.4	0.2	setosa	0
2	4.7	3.2	1.3	0.2	setosa	0
3	4.6	3.1	1.5	0.2	setosa	0
4	5.0	3.6	1.4	0.2	setosa	0

 # Stratification is important to balance classes between split

```
X = iris_df.iloc[:,:4].values
y = iris_df["target"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3,
    stratify=y,
    random_state=42
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

Output:
```
(105, 4) (45, 4) (105,) (45,)
```

Scaling the values

```
scaler = StandardScaler()
scaler
```

Output:
```
StandardScaler()
```

Apply scaling to training and test data

```
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print("Values scaled with Standard Scaler")
```

Output:
```
Values scaled with Standard Scaler
```

View original training data (before scaling)

```
X_train[:5]
```

Output:
```
array([[5.1, 2.5, 3. , 1.1],
[6.2, 2.2, 4.5, 1.5],
[5.1, 3.8, 1.5, 0.3],
[6.8, 3.2, 5.9, 2.3],
[5.7, 2.8, 4.1, 1.3]])
```

View scaled training data (after scaling)

```
X_train_s[:5]
```

Output:
```
array([[-0.90045861, -1.22024754, -0.4419858 , -0.13661044],
[ 0.38036614, -1.87955796,  0.40282929,  0.38029394],
[-0.90045861,  1.63676428, -1.2868009 , -1.17041921],
[ 1.07899781,  0.31814344,  1.19132338,  1.41410271],
[-0.20182693, -0.56093712,  0.17754527,  0.12184175]])
```

Binarizing encoded labels for ROC-AUC metric calculation

```
y_test_bin = label_binarize(y_test, classes=[0,1,2])
y_test_bin[:5]
```

Output:

```
array([[0, 0, 1],
[0, 1, 0],
[0, 0, 1],
[0, 1, 0],
[0, 0, 1]])
```

Step 5: Model Training

Model Training

```
# k = 10 (Heuristic Rule)
model = KNeighborsClassifier(n_neighbors=10)
model.fit(X_train_s, y_train)
```

Output:

```
KNeighborsClassifier(n_neighbors=10)
```

Step 6: Model Evaluation

Model Evaluation

```
y_pred = model.predict(X_test_s)
y_pred
```

Output:

```
array([2, 1, 1, 1, 2, 2, 1, 1, 0, 2, 0, 0, 2, 2, 0, 2, 1, 0, 0, 0, 1, 0,
1, 2, 1, 1, 1, 1, 1, 0, 2, 2, 1, 0, 2, 0, 0, 0, 0, 1, 1, 0, 1, 2,
1])
```

Get prediction probabilities

```
y_prob = model.predict_proba(X_test_s)
y_prob[:5]
```

Output:

```
array([[0. , 0.1, 0.9],
[0. , 0.9, 0.1],
[0. , 0.7, 0.3],
[0. , 0.7, 0.3],
[0. , 0.4, 0.6]])
```

Generate classification report

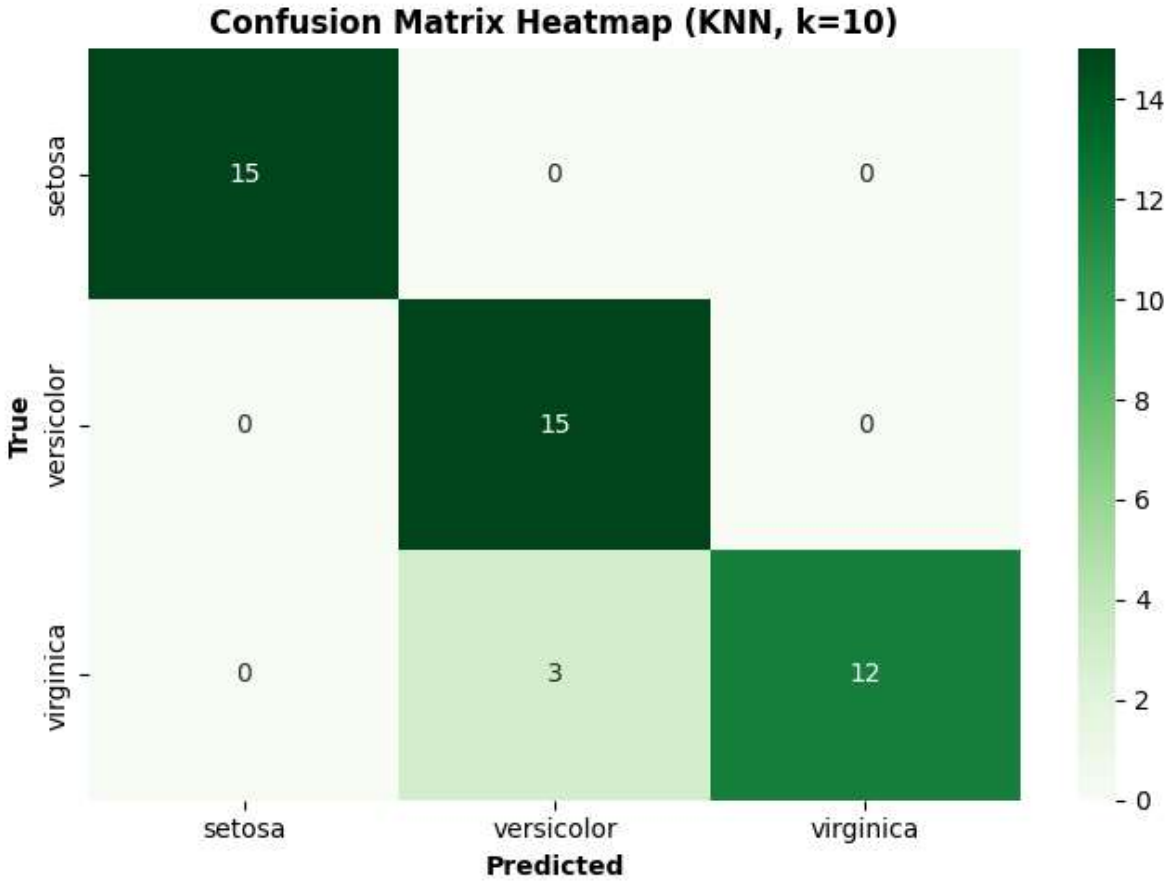
```
print("\\nClassification Report (k=10):")
print(classification_report(
    y_test,
    y_pred,
    target_names=target_names,
    digits=4
))
```

Output:

```
Classification Report (k=10):
precision    recall  f1-score   support
setosa     1.0000    1.0000    1.0000        15
versicolor     0.8333    1.0000    0.9091        15
virginica     1.0000    0.8000    0.8889        15
accuracy                         0.9333        45
macro avg     0.9444    0.9333    0.9327        45
weighted avg     0.9444    0.9333    0.9327        45
```

Visualize confusion matrix heatmap

```
plt.figure(figsize=(7, 5))
sns.heatmap(
    confusion_matrix(y_test, y_pred), annot=True, fmt="d", cmap="Greens",
    xticklabels=target_names, yticklabels=target_names
    )
plt.xlabel("Predicted", fontweight='bold')
plt.ylabel("True", fontweight='bold')
plt.title("Confusion Matrix Heatmap (KNN, k=10)", fontweight='bold')
plt.tight_layout()
plt.show()
```



Step 7: Model Simulation

 # Final Metrics

```
# Accuracy will be same as F1 score as dataset is perfectly balanced and stratified
print(f'''Accuracy   : {accuracy_score(y_test, y_pred)}
Precision  : {precision_score(y_test, y_pred, average="macro")}
Recall     : {recall_score(y_test, y_pred, average="macro")}
F1         : {f1_score(y_test, y_pred, average="macro")}
ROC        : {roc_auc_score(y_test_bin, y_prob, multi_class="ovr", average="macro")}
''')
```

Output:

```
Accuracy   : 0.9333333333333333
Precision  : 0.9444444444444445
Recall     : 0.9333333333333332
F1         : 0.9326599326599326
ROC        : 0.9948148148148149
```